

Pillar ③ — The Agent / Automation Factory

July 17, 2026

CONTENTS

name: pillar-3-agents description: > Pillar ③ of the AI Infrastructure Protocol — the agent / automation factory. Takes a single backlog i

Pillar ③ — The Agent / Automation Factory

Where this sits in the journey

Reuse before build (non-negotiable)

Step 1 — Intake: the agent spec

Step 2 — Taxonomy: what you're building, and with which skill

① Chat / assistant agents

② Tool / integration automations

③ Browser / RPA automations

④ Event-driven hooks

⑤ Multi-step orchestrations

Content-generation micro-automations (a note)

Artifact-type → skill map (the lookup)

Step 3 — The build loop

PROPOSE

BUILD

VERIFY — does it actually perform the task on real inputs?

REGISTER

Step 4 — The Security gate is mandatory (G3)

Step 5 — Guardrails (baked into every artifact, not bolted on)

Full flow — blueprint item to shipped agent

Worked example — TODO: build the first real one

Quick reference

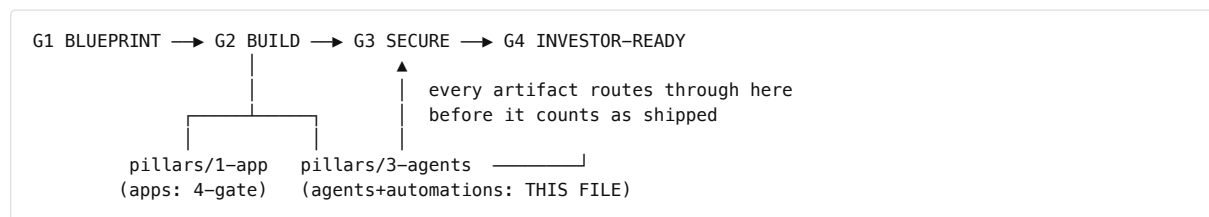
name: pillar-3-agents description: > Pillar ③ of the AI Infrastructure Protocol — the agent / automation factory. Takes a single backlog item from the blueprint (pillar ②) and turns it into a working, verified agent or automation, registered in .protocol/artifacts.json and routed through the ⑤ Security gate before it counts as shipped. Use during G2 BUILD for every backlog artifact that is NOT a full app (apps go to pillars/1-app → 3day-protocol). Use when asked to “build the agent”, “wire this automation”, “make this run without a human”, or “clear an agent to ship”.

Pillar ③ — The Agent / Automation Factory

You are the operator of a factory. Raw material in: one backlog item from `blueprint.json` with a disposition of **augment-ai** or **fully-automate**. Finished good out: a running artifact that performs the task on real inputs, is registered, and has passed Security. Everything here is an instruction, not a suggestion. When the factory and your instinct disagree, the factory wins.

This pillar owns the **G2 build path for non-app artifacts**. Apps go through the four-gate app engine (`pillars/1-app` → `3day-protocol/`). Everything else — chat agents, integration automations, browser/RPA jobs, event hooks, multi-step orchestrations — is built here. The dividing line is simple: if it has its own authenticated UI and database, it is an app; otherwise it is an agent/automation and belongs in this factory.

Where this sits in the journey



The factory does not decide *whether* to automate a process — the blueprint already did that (impact × effort, AI-involvement score). The factory decides *how*, builds it, and proves it works.

Reuse before build (non-negotiable)

Eight skills are bundled under `/Users/ianveber/Desktop/Cloude CODE/ai-infrastructure-protocol/skills/agents/`. You compose them; you do not reinvent them. Writing a bespoke integration when `composio-cli` already exposes the tool slug is a factory violation — log it with a reason or don't do it. Read the target skill's own `SKILL.md` before you build; the mappings below tell you which one to open.

<code>skills/agents/</code>	
<code>├ whatsapp-ai-agent/</code>	chat/assistant agents on WhatsApp (Manychat+Make+Claude)
<code>├ voice-builder/</code>	persona/voice profile that any chat agent speaks in
<code>├ composio-cli/</code>	tool/integration automations across 250+ apps
<code>├ agent-browser/</code>	browser/RPA automation (accessibility-tree driven)
<code>├ scrape/</code>	read-only data extraction from web pages
<code>├ hook-generator/</code>	content-generation micro-automation (marketing artifacts)
<code>├ dispatching-parallel-agents/</code>	fan-out orchestration (N independent sub-tasks)
<code>└ pair-agent/</code>	hand a scoped browser session to a remote agent

Step 1 — Intake: the agent spec

Before building anything, convert the backlog item into an **agent spec** and write it to `.protocol/agents/<slug>/spec.json`. The blueprint gives you the *what*; the spec pins the *contract* the built artifact must satisfy and the verify gate will check.

An agent spec has exactly these fields. A missing field is not a shortcut — it is a question you owe the client before you build.

Field	Meaning	Example
slug	kebab-case id, unique in the run	invoice-chaser
blueprintRef	id of the backlog item this satisfies	BL-014
type	one taxonomy type (see Step 2)	integration-automation
trigger	what starts a run — event / schedule / message / manual	schedule: daily 08:00 Europe/Ljubljana
inputs	data the artifact consumes, with source + shape	unpaid invoices from Stripe (id, amount, dueDate, customerEmail)
actions	ordered steps it performs	1. fetch overdue 2. draft reminder 3. queue for approval
tools	integrations/credentials it touches, each with a scope	stripe:read, gmail:draft
outputs	what it produces + where it lands	draft emails in Gmail Drafts + row in log sheet
guardrails	the hard limits (see Step 5)	never send; drafts only; ≤50 runs/day; €2/day cost cap
hitl	human-in-the-loop points — where a person must approve	send step: human approves each draft
verify	the acceptance test — a concrete real-input check	given 3 seeded overdue invoices, produces 3 correct drafts, sends 0

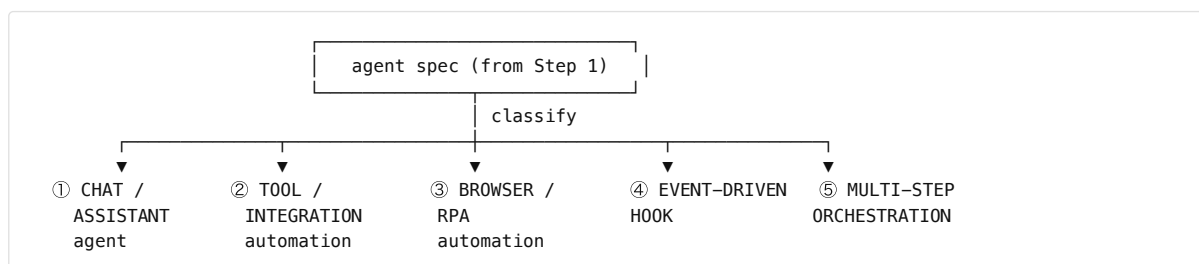
Rules for a valid spec:

- Every outward or destructive action has an `hitl` gate OR an explicit written waiver.** Sending email, posting publicly, moving money, deleting data, writing to a system of record — these default to human-approval. “Fully automate” in the blueprint means the *drafting* is automated, not that the safety is removed. If the client genuinely wants send-without-approval, that is a decision recorded in the spec with their name on it, and Security (G3) will see it.
- `tools` are least-privilege by construction.** Request the narrowest scope that makes the actions work: `read` before `write`, `draft` before `send`, one toolkit’s one operation before broad API access. Over-scoped credentials are a G3 finding.
- `verify` is a test, not a wish.** It names real (or realistically seeded) inputs and the exact observable output. “Works well” is not a verify criterion. “Given these 3 inputs, produces these 3 outputs and takes zero outward actions” is.

Close intake: the spec is written and internally consistent (every action’s tool is in `tools`; every outward action has an `hitl` entry or waiver). This is the contract.

Step 2 — Taxonomy: what you’re building, and with which skill

Classify the spec into exactly ONE type. The type decides the builder skill and the verify method. If a backlog item spans two types, split it into two specs — a chat agent that also runs a nightly sync is a chat agent *and* a scheduled automation, built and verified separately, composed at the end.



① CHAT / ASSISTANT AGENTS

An LLM that talks to a human or another system and answers/acts in natural language.

- **Builder:** `whatsapp-ai-agent` (WhatsApp support/sales bot via Manychat + Make.com
 - Anthropic API). For the agent's *voice/persona*, run `voice-builder` first and feed its `voice.md` into the system prompt so the bot speaks as the company, not as generic AI.
- **Signals:** trigger is an inbound message; primary output is a reply; the value is conversation.
- **Guardrail note:** support bots default to **answer-only**; any action that writes to a system (create ticket, issue refund, book slot) is an `hitl` approval step, not an autonomous one, until Security clears it.

② TOOL / INTEGRATION AUTOMATIONS

Move data and take actions across SaaS apps — Gmail, Stripe, GitHub, Sheets, Slack, Notion, Calendar, and 250+ others.

- **Builder:** `composio-cli`. Discover the slug with `composio search "<task>"`, connect least-privilege with `composio link <toolkit>`, inspect with `--get-schema`, preview with `--dry-run`, then `composio execute` or `composio run` for scripted multi-tool logic. `--dry-run` is the factory's best friend: it proves the request shape without performing the action.
- **Signals:** trigger is a schedule or an upstream data change; the value is data moved / a record created / a message routed.
- **Guardrail note:** outward `execute` calls (send, create, delete) stay behind an approval queue or an `hitl` gate; read/fetch/search calls may run autonomously.

③ BROWSER / RPA AUTOMATIONS

Drive a real browser for apps that have no API — legacy portals, government sites, internal tools, vendor dashboards.

- **Builder:** `agent-browser` for mutating flows (login, fill, click, submit, screenshot-verify); `scrape` for read-only extraction. `agent-browser` is accessibility-tree driven with `@eN` element refs — load its live workflow with `agent-browser skills get core` before writing commands.
- **Signals:** the target has no API; the task is “do what a person does in this web UI”.
- **Guardrail note:** browser automations run against real sessions — scope the login to a dedicated service account, never the client's personal SSO, and gate any irreversible click (submit, pay, delete) behind `hitl`.

④ EVENT-DRIVEN HOOKS

Fire deterministic logic the moment something happens — a webhook lands, a file changes, a Composio trigger emits, a git event occurs.

- **Builder:** `hook-generator` for lifecycle/tooling hooks; `composio listen` (from `composio-cli`) to subscribe to app trigger events (new email, new Stripe charge, new GitHub issue) and run an action snippet in response.
- **Signals:** trigger is an event, not a schedule or a message; the reaction is immediate and rule-shaped.
- **Guardrail note:** hooks run unattended by definition — the cost cap and the outward-action `hitl` gate matter *most* here, because there is no human in the loop at fire time. A runaway hook with send permission is the worst failure mode in the factory.

⑤ MULTI-STEP ORCHESTRATIONS

One goal that decomposes into several sub-agents or a chained pipeline.

- **Builder:** `dispatching-parallel-agents` when the sub-tasks are independent (fan out one agent per problem domain, each with isolated context); `composio run` for a sequential chain that plumbs one tool's output into the next; `pair-agent` when a step must hand a scoped browser session to a remote/other agent.
- **Signals:** the spec's `actions` list is long and branches; no single skill covers it end-to-end.
- **Guardrail note:** an orchestration inherits the *union* of its steps' scopes — which makes it the highest-privilege artifact type. Decompose so each sub-agent holds only its own least-privilege credentials; never give the orchestrator one god-token.

CONTENT-GENERATION MICRO-AUTOMATIONS (A NOTE)

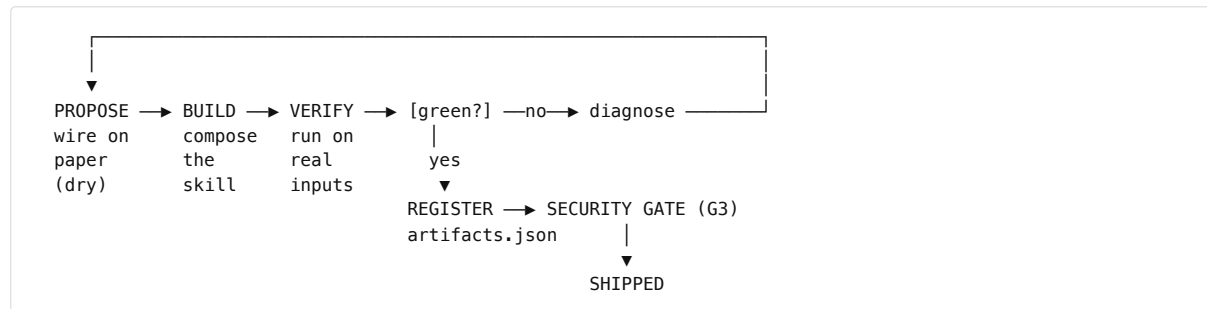
`hook-generator` and `voice-builder` also serve pillar ④/marketing as pure content producers (hooks, voice profiles). In the factory they are **components**, not standalone artifacts: `voice-builder` supplies the persona a chat agent speaks in; `hook-generator` can be wrapped by an orchestration that posts on a schedule. A content generator only becomes a registered artifact when it has a trigger and an output destination — otherwise it is a prompt, not an automation.

ARTIFACT-TYPE → SKILL MAP (THE LOOKUP)

Spec type	Primary skill(s)	Verify method
chat-agent	whatsapp-ai-agent (+ voice-builder for persona)	send N real messages → correct replies, zero un-gated actions
integration-automation	composio-cli (execute / run / proxy)	run against seeded data → records/messages match spec, --dry-run clean
browser-rpa	agent-browser (mutating) / scrape (read-only)	replay the flow on the live site → screenshot/extract matches expected
event-hook	hook-generator / composio listen	emit a real trigger event → correct reaction fires exactly once
orchestration	dispatching-parallel-agents / composio run / pair-agent	end-to-end run on real inputs → final output matches spec

Step 3 — The build loop

For each spec, run the loop until verify is green. Log each pass in `.protocol/agents/<slug>/journal.jsonl`.



PROPOSE

State, in the journal, how you will build it: which skill, which tool slugs / hooks / sub-agents, and the smallest safe way to prove the request shape. For integration automations, this is where you run `composio execute <slug> --get-schema` and `--dry-run` — you propose against the real schema, not your memory of it. Nothing outward or destructive happens in PROPOSE.

BUILD

Compose the mapped skill(s) into the artifact. Wire trigger → inputs → actions → outputs exactly as the spec says. Inject `hitl` gates at every outward/destructive step now — retrofitting a safety gate after verify is how they get skipped. Store the built artifact and its config under `.protocol/agents/<slug>/` (or the tool's native location, referenced from there).

VERIFY — DOES IT ACTUALLY PERFORM THE TASK ON REAL INPUTS?

This is the gate on the build, and it is behavioral, not aspirational. Run the spec's `verify` criterion:

- **Real (or realistically seeded) inputs**, never toy stubs. If the task is “chase overdue invoices”, seed three overdue invoices and run it.
- **Observe the actual output**, not the module's green checkmark. (Hard-won lesson from `whatsapp-ai-agent`: Make.com shows all modules green while the payload is malformed — you must inspect Module 3's real output. The same discipline applies to every artifact: read the produced artifact, not the runner's exit code.)
- **Assert the guardrails held during the run**: outward actions that should have been gated produced drafts/queue-entries, not sends; the cost stayed under cap; scopes weren't exceeded.
- **Idempotency / exactly-once for hooks**: emit the trigger twice; confirm the reaction fires the right number of times, not N times.

Write the evidence to `.protocol/agents/<slug>/verify.json`:

```
{
  "slug": "invoice-chaser",
  "ranAt": "2026-07-17T09:12:00+02:00",
  "inputs": "3 seeded overdue invoices (INV-1,INV-2,INV-3)",
  "expected": "3 correct reminder drafts, 0 sends",
  "observed": "3 drafts in Gmail Drafts, 0 messages sent, cost €0.02",
  "guardrails": { "outwardGated": true, "costUnderCap": true, "scopeRespected": true },
  "result": "green"
}
```

VERIFY is red until the observed output matches expected AND every guardrail held. A red verify never advances. If verify tooling is genuinely unavailable, that is a red — never a silent skip (same rule the app engine applies to a missing Docker/CLI).

REGISTER

Only a green verify earns a row in `.protocol/artifacts.json`. This is the shared registry the whole protocol reads (G2 completeness, G3 security scope, G4 investor story). Append (do not overwrite) an entry:

```
{
  "id": "invoice-chaser",
  "pillar": "3-agents",
  "type": "integration-automation",
  "blueprintRef": "BL-014",
  "builtWith": ["composio-cli"],
  "trigger": "schedule: daily 08:00 Europe/Ljubljana",
  "tools": ["stripe:read", "gmail:draft"],
  "outputs": "Gmail drafts + log sheet row",
  "hitl": ["send step: human approves each draft"],
  "verify": ".protocol/agents/invoice-chaser/verify.json",
  "verifyResult": "green",
  "security": "pending",
  "shipped": false
}
```

`security: "pending"` and `shipped: false` are correct at registration. **A verified artifact is not a shipped artifact.** It becomes shipped only when the ⑤ Security gate flips `security` to `"green"`.

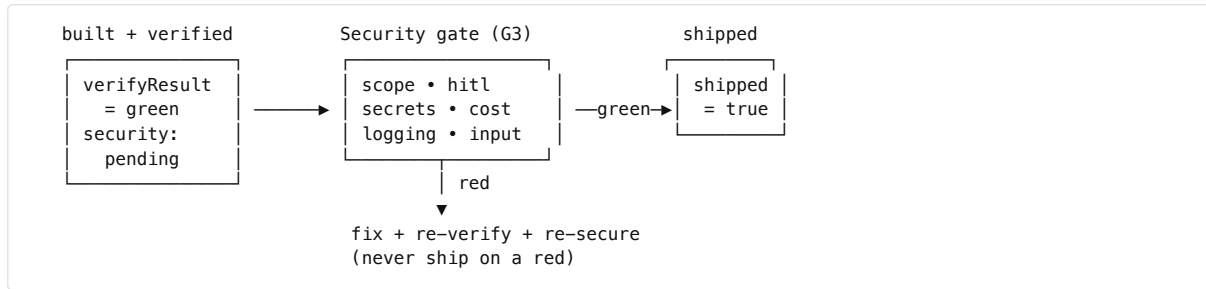
Step 4 — The Security gate is mandatory (G3)

Every agent MUST route through pillar ⑤ Security before it counts as shipped. A green verify proves it *works*; the Security gate proves it is *safe to run unattended in a real company*. Verify without Security is a loaded gun with the safety off.

Hand the artifact to `pillars/5-security/SKILL.md`. The security pass reads `artifacts.json`, and for each agent/automation checks (agent-specific, on top of the app-engine checks it reuses for apps):

- **Credential scope** matches the spec's least-privilege claim — no token broader than its `tools` list. Broader = RED.
- **Outward/destructive actions are gated** by the `hitl` entries the spec promised — an un-gated send/pay/delete found in the built artifact is RED, regardless of what the spec said.
- **Secrets are not embedded** in the artifact config, prompt, or logs (API keys, Composio tokens, webhook secrets). Any hit is RED, redacted in evidence.
- **Cost cap is enforced in the artifact**, not just written in the spec — an event hook with no rate limit / spend ceiling is RED (the worst case: an unattended hook with send permission and no cap).
- **Logging is on** — every run leaves an auditable trail (who/what/when/cost).
- **Input is validated** at the trigger boundary — webhook/message payloads are parsed and bounded before use, not passed through raw.

The security skill writes the result to `.protocol/gates/gate-3.json` and flips each artifact's `security` field. **Only when `security: "green"` do you set `shipped: true`.** A missing tool or an unproven check is RED, never a skip. This mirrors the app engine's iron rule: nothing ships that Security hasn't seen.

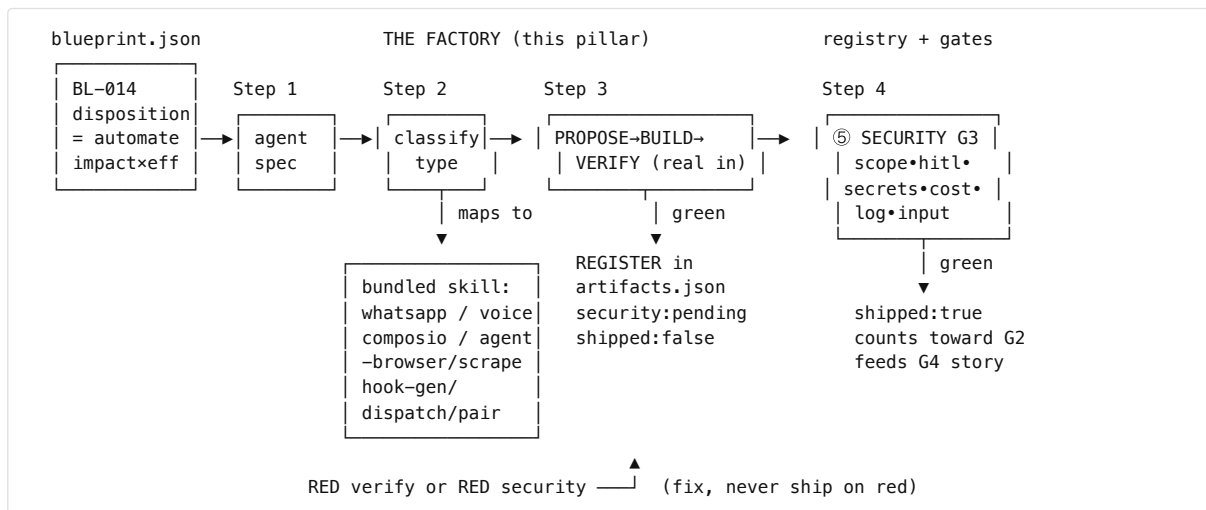


Step 5 — Guardrails (baked into every artifact, not bolted on)

These are the factory’s standing orders. They live in the spec, are wired in BUILD, asserted in VERIFY, and enforced in the Security gate. Four of them:

- Least-privilege credentials.** Request the narrowest scope that works, per toolkit, per action. `read` before `write`, `draft` before `send`. Use a dedicated service account, never the client’s personal login. One artifact = one scoped credential set; orchestrations hold no god-token (each sub-agent holds its own). *Enforced:* G3 compares granted scope to the spec’s `tools`.
- Human-approval gates for destructive/outward actions.** Any action that leaves the company or cannot be undone — send email, post publicly, move money, delete a record, write to a system of record — defaults to producing a **draft or a queued proposal a human approves**, not an autonomous execution. “Fully automate” automates the drafting; a named human owns the send. Removing an approval gate is a decision recorded in the spec with a name attached. *Enforced:* G3 fails on an un-gated outward action in the built artifact.
- Logging.** Every run writes an auditable line: trigger, inputs (bounded/redacted), actions taken, outputs produced, cost. No silent runs. This is also the raw material for pillar ④’s investor story (throughput, hours saved) — an unlogged automation can’t be counted. *Enforced:* G3 fails if the artifact has no run log.
- Cost caps.** Every artifact declares a spend ceiling (per-run and per-day) and a run-rate limit. Event hooks and orchestrations especially — they run without a human at the trigger. Exceeding the cap halts the artifact, it does not silently keep spending. *Enforced:* G3 fails on a hook/automation with no enforced cap.

Full flow — blueprint item to shipped agent



Gate 2 (BUILD) is green for this pillar when **every backlog item marked “this sprint” is built, verified, registered, AND shipped: true** — which means it has already cleared G3. An artifact stuck at `security: pending` keeps Gate 2 red. That is the point: the factory does not get to declare victory on “it works”; it declares victory on “it works, it’s safe, and it’s proven.”

Worked example — TODO: build the first real one

The methodology above is complete. The one thing this file cannot fabricate is a live, credential-backed run. Build the first real artifact end-to-end and paste its real evidence here so the pillar ships with a proof, not a template.

TODO(live-example): Pick the highest-ranked `integration-automation` from a real client blueprint (the `invoice-chaser` shape is a strong candidate — Stripe read + Gmail draft, one outward action fully gated). Then:

1. **TODO:** Write the real `.protocol/agents/<slug>/spec.json` from the backlog item.
2. **TODO:** `composio search "<the task>"` → record the real tool slugs.
3. **TODO:** `composio link stripe` / `composio link gmail` with least-privilege scopes → record which scopes were actually granted.
4. **TODO:** `composio execute <slug> --get-schema` and `--dry-run` → paste the real request shape.
5. **TODO:** Seed real (or sandbox) inputs, run the artifact, and capture the real `verify.json` (observed output + guardrail assertions).
6. **TODO:** Append the real `artifacts.json` entry.
7. **TODO:** Run `pillars/5-security/SKILL.md` against it and paste the real `gate-3.json` result + the `security: green` / `shipped: true` flip.

Until that block is filled with real output, this pillar is methodology-complete but proof-pending — the same honesty standard the whole protocol holds itself to: never claim, always prove.

Quick reference

Situation	Do this
New backlog item to automate	Step 1 — write <code>.protocol/agents/<slug>/spec.json</code>
Which skill builds it?	Step 2 — classify type, use the artifact-type → skill map
WhatsApp/chat bot	<code>whatsapp-ai-agent</code> (+ <code>voice-builder</code> for persona)
Move data between SaaS apps	<code>composio-cli</code> (<code>search</code> → <code>link</code> → <code>--get-schema</code> → <code>--dry-run</code> → <code>execute</code> / <code>run</code>)
Automate a web UI with no API	<code>agent-browser</code> (mutating) / <code>scrape</code> (read-only)
React to an event immediately	<code>hook-generator</code> / <code>composio listen</code>
One goal, many sub-tasks	<code>dispatching-parallel-agents</code> / <code>composio run</code> / <code>pair-agent</code>
"Is it done?"	verify green + registered + <code>security: green</code> + <code>shipped: true</code>
Verify tooling is down	RED, never a skip — fix the tooling or don't ship
Client wants send-without-approval	record it in the spec with their name; G3 still sees it